Learning to Route Under Load: An Inductive Graph Network that Amortizes Congestion-Aware Traffic Engineering for LEO Mega-Constellations

Phuc Hao Do, Nang Hung Van Nguyen, and Minh Tuan Pham

***Abstract*—Low Earth orbit (LEO) mega-constellations carry traffic over a dense mesh of inter-satellite links whose topology, although time varying, is known in advance from orbital ephemeris. Under load, routing on propagation delay alone concentrates traffic on a few bottleneck links while alternative paths sit idle, wasting most of the achievable performance: in our study it inflates total travel time by up to a factor of forty. The congestion-aware optimum, the user equilibrium, removes this waste but requires the full demand matrix and an iterative solve at every slot, which does not scale to constellations of more than a thousand satellites with rapidly changing topology. We formalize routing under load on LEO as the amortization of a traffic-assignment equilibrium, decompose it into a price-prediction sub-problem, a decoding sub-problem, an inductive-transfer sub-problem, and a proactive sub-problem, and solve the first with an inductive graph neural network that predicts the equilibrium congestion price on each inter-satellite link in a single forward pass from cheap, observable load features. On a constellation simulator with a standard Bureau of Public Roads cost model, the network closely approaches the user equilibrium in distribution (recovering about 97% of the gain over blind routing), recovers about 94% of that gain when transferred zero shot to a shell twelve times larger, runs 21 to $29\times$ faster than the iterative solve with the saving growing with scale, and, fed predicted demand, sustains low travel time under drifting hotspots where a reactive controller degrades to $1.6\times$ blind. We contribute, in addition, a record of three evaluation pitfalls that silently fabricate or distort such results, with a protocol to avoid them, and we report a negative result: a quantum-walk propagation operator that motivated this study offers no benefit once load features are available, and a plain graph convolution is preferable.**

***Index Terms*—LEO satellite networks, non-terrestrial networks, routing, traffic engineering, graph neural networks, congestion control, user equilibrium, inductive learning, evaluation methodology.**

I. Introduction

LOW Earth orbit (LEO) mega-constellations such as Starlink and Kuiper interconnect their satellites with laser inter-satellite links (ISLs) arranged in a regular grid, forming a wide-area packet network that moves with the constellation. Two features distinguish this network from a terrestrial one. First, the topology is time varying but *deterministic*: the position of every satellite, hence every ISL and its propagation delay, is known in advance from the orbital parameters. Second, the network is large and fast: a single shell contains a thousand or more satellites, and cross-plane links open and close as satellites traverse the constellation.

Routing such a network on propagation delay alone is attractive because the shortest path is cheap and, when lightly loaded, near optimal. Under load it is wasteful: it funnels many flows onto the same minimum-hop links, creating hotspots, while geometrically longer but idle paths are ignored. We measure this waste directly (Section VIII-A) and find that, once the network is moderately loaded, congestion-aware routing cuts total travel time by more than half, and by up to a factor of forty under heavy load.

The congestion-aware optimum is classical. Routing every flow so that none can unilaterally lower its cost yields the *user equilibrium* (UE), computed by iterating shortest-path assignment against a load-dependent cost. UE is the right target but expensive: it needs the entire demand matrix and many shortest-path rounds per slot. At constellation scale, with topology changing every few seconds, recomputing UE from scratch each slot is impractical.

This tension, a cheap-but-blind heuristic versus an expensive-but-optimal solve, motivates our approach: *amortize* the equilibrium. A graph neural network (GNN) observes cheap features (per-node traffic intensity and the load that one congestion-blind pass would induce) and predicts, in a single forward pass, the equilibrium congestion price on every ISL. Routing on the predicted cost recovers most of the congestion-aware gain at a fraction of the cost of solving UE. Because the model's parameters do not depend on the number of satellites, it is *inductive*: trained on a small shell, it applies unchanged to a larger one. And because the demand pattern drifts predictably (the dense-traffic band rotates under the constellation), feeding the predicted near future demand lets the router act *proactively*, before congestion forms. Figure 1 summarizes the pipeline.

To make the stakes concrete, consider the dense-traffic band passing over a populated longitude. Delay-optimal routing sends the surge of flows along the few minimum-hop ISL chains that cross that region, driving a handful of links far past capacity while the cross-plane links a hop away carry almost nothing. With a fourth-power congestion cost, the travel time on those saturated links explodes, and the network appears to be in overload even though the same traffic spread across the available ISLs would be comfortably feasible. An operator that recomputes the equilibrium each slot fixes this but pays a heavy, repeated optimization; an operator that routes on stale

P. H. Do is with Danang Architecture University, Da Nang, Viet Nam (e-mail: haodp@dau.edu.vn).

N. H. V. Nguyen and M. T. Pham are with the University of Science and Technology, Da Nang, Viet Nam (e-mail: nguyenvan@dut.udn.vn; pmtuan@dut.udn.vn).

Corresponding author: Minh Tuan Pham.

This research is funded by the Science and Technology Development Fund of The University of Danang under project number B2025-DN02-26.

or blind information does not fix it at all. The method in this paper gives a third option: spend the optimization once, offline, and let a network predict its outcome thereafter.

A second, methodological thread runs through the paper. Learned traffic engineering is easy to evaluate incorrectly: a greedy baseline that silently drops hard flows, a scaling claim that rests on a ratio with a moving denominator, or an equilibrium reference measured off its operating point can each fabricate or distort a result. We encountered all three, caught them, and distil a protocol (Section VI) so that the reported numbers are trustworthy and reproducible.

Contributions.

- We formalize routing under load on LEO as the amortization of a traffic-assignment equilibrium and *decompose* it into four sub-problems (Section IV), each mapped to a measurable claim.
- **C1 (amortization).** A one-shot GNN closely approaches the user equilibrium in distribution, recovering about 97% of the gain over blind routing, at 21 to $29\times$ less computation than the iterative solve.
- **C2 (inductive scale generalization).** Trained on a 132-satellite shell, the same model transferred zero shot to a 1584-satellite shell ($12\times$ larger) recovers about 94% of the gain and remains the best *deployable* policy.
- **C3 (cost that improves with scale).** We give a complexity analysis and wall-clock measurements showing the inference saving over UE *grows* with the constellation, reaching $29\times$ at 1584 satellites.
- **C4 (proactive).** Under drifting hotspots, routing on predicted demand tracks the clairvoyant optimum, whereas a reactive one-slot-lagged controller degrades below blind shortest path as the drift grows.
- **Evaluation methodology.** We document three pitfalls that silently distort results in this setting, with fixes, and verify the corrected estimator against the price-of-anarchy ordering $\mathrm{TTT(SO)} \leq \mathrm{TTT(UE)}$.
- **Honest negative result.** The study began as a quantum-walk GNN; we report the kill-gate record and why a plain graph convolution is the right operator here.

II. Related Work

LEO routing and topology. The deterministic, time-varying structure of LEO ISL networks has been studied for low-latency routing and topology design [1], [2]. Handley showed that a +Grid of ISLs can beat terrestrial fiber on long paths because light is faster in vacuum, making the ISL layer worth routing carefully [1]; Bhattacherjee and Snoeren explored how the choice of which satellites to interconnect shapes path length and churn [2]. The broader push toward non-terrestrial networks in the 6G era is surveyed in [17], [18], and the design space of competing LEO constellations is compared in [19]; measurement studies of operational systems such as Starlink confirm that capacity, not only geometry, shapes the user experience [20]. Packet-level simulators such as Hypatia [3] and StarPerf [4] reconstruct the moving topology from constellation parameters and have driven work on path stability, handover, and delay under realistic orbital dynamics. This line predominantly routes on delay or hop count and evaluates a lightly loaded network; our contribution is orthogonal and complementary, namely how to route when the bottleneck is congestion rather than geometry, and how to do so without a per-slot optimization.

GNNs for networking. Message passing [21], graph convolutions [5], attention [6], and inductive sampling [7] underpin learning on graphs, whose expressive power [22] and relational inductive biases [23] are now well understood. In networking specifically, GNNs have been used for modeling and control: RouteNet predicts per-path delay and jitter for network modeling and what-if analysis [8]; surveys map the emerging use cases [24]; and learned routing has been pursued by generating distributed protocols [25], by data-driven distance-vector routing [26], and by reinforcement learning with GNNs [9]. These approaches either model performance or learn a control policy by trial and error. Ours is supervised and predicts a congestion-price *field* that is decoded by a single one-shot pass, which keeps inference to one forward evaluation and makes inductivity across constellation size immediate.

Traffic engineering and equilibria. Load-dependent link cost, the user equilibrium, and the system optimum are classical in transportation science [10], [11], [12], [27]: Wardrop's principles characterize the equilibrium, the Beckmann transformation casts it as a convex program solvable by Frank-Wolfe and the method of successive averages [28], [27], and the BPR function is the standard congestion cost. The gap between selfish and optimal routing is the price of anarchy, bounded for such cost classes by Roughgarden and Tardos [13]. In data networks, traffic engineering has a long line of work, from tuning OSPF link weights [29] and responsive-yet-stable load balancing [30] to software-driven wide-area networks [31], [32]. These solve an optimization per measurement epoch; the novelty in LEO is that the topology is large and changes every few seconds, so a per-slot solve is the bottleneck and amortizing it is the prize.

Learning-based control. Deep reinforcement learning has been applied to resource management [33], datacenter traffic optimization [34], and routing over a fixed topology [14], [15]. Such policies are powerful but require online exploration and are tied to the training topology. We instead supervise on a computable equilibrium and exploit the predictable demand drift for proactivity, which removes exploration at deployment and makes the learned map transfer across constellation size.

Quantum walks and learned operators. Continuous-time quantum walks have been used as a graph propagation operator to capture long-range structure [16], and recent function-learning layers such as Kolmogorov-Arnold networks [35] offer alternative inductive biases. We evaluate a quantum-walk operator as a candidate and report it as a negative ablation (Section VIII-I).

III. System Model

Table I lists the notation.

A. Time-varying constellation graph

A Walker shell of S satellites in P planes is propagated on circular orbits. Each satellite holds two intra-plane ISLs

TABLE I
NOTATION.

symbol	meaning
$G_t = (V, E_t)$	constellation graph at slot t
prop_{uv}	propagation delay of ISL (u, v)
cap	ISL capacity
$\mathcal{D} = \{(s, d, r)\}$	demand set (source, destination, rate)
x_{uv}	load (aggregate rate) on ISL (u, v)
$\mathrm{cost}_{uv}(x)$	realized link cost under load (BPR)
g^*_{uv}	user-equilibrium congestion price of (u, v)
$\hat{g}_{uv}$	price predicted by the GNN
$h_v^{(l)}$	embedding of node v at layer l
$\mathrm{TTT}(\pi)$	total travel time of routing policy π

and up to two cross-plane ISLs, forming a +Grid (Fig. 3). The topology over a horizon is a known sequence $\{G_t\}_{t=0}^{T-1}$, $G_t = (V, E_t)$, with $|V| = S$. Each ISL $(u, v) \in E_t$ has a propagation delay $\mathrm{prop}_{uv} = \|p_u - p_v\|/c$ from the known positions p, and a capacity cap. Cross-plane links above a polar cutoff latitude are deactivated, as in operational designs; for a 53° shell the cutoff is never reached, so the ISL graph is near static and the dynamics enter through the demand.

B. Demand, flows, and link load

A demand set $\mathcal{D} = \{(s_k, d_k, r_k)\}$ requests rate r_k from s_k to d_k. We draw demands from a gravity model: a node's attractiveness is $w_v \propto \exp(\kappa \cos(\angle v - \phi_t))$, peaked on a longitude band ϕ_t, and endpoints are sampled with probability proportional to $w_s w_d$. The band ϕ_t rotates between slots, a proxy for the dense-population and sub-solar region rotating under the constellation as Earth turns; the per-slot drift $\Delta = \phi_{t+1} - \phi_t$ is known from the same ephemeris that fixes the topology. This makes the demand non-stationary (the hotspot moves) yet predictable (its motion is known), which is exactly the structure the proactive variant exploits. Offered load is controlled by the number of demands, which we sweep to span the lightly loaded and heavily congested regimes. A routing policy maps each demand k to a path P_k in G_t. Writing r_k for the rate on P_k, the load on an ISL is

$$x_{uv} = \sum_k r_k \mathbb{1}[(u, v) \in P_k], \tag{1}$$

subject to flow conservation at every node $v \notin \{s_k, d_k\}$,

$$\sum_{u:(u,v)\in E_t} f^k_{uv} = \sum_{w:(v,w)\in E_t} f^k_{vw}, \tag{2}$$

with f^k the per-demand flow.

C. Congestion cost and realized travel time

The realized cost of a link under load follows the Bureau of Public Roads (BPR) form [11],

$$\mathrm{cost}_{uv}(x) = \mathrm{prop}_{uv}\Big(1 + \alpha\,(x_{uv}/\mathrm{cap})^\beta\Big), \quad \alpha = 0.15,\ \beta = 4, \tag{3}$$

which is smooth, strictly increasing, and convex in x, and which blows up as a link approaches and exceeds capacity. The total travel time of a policy π inducing loads $x(\pi)$ is

$$\begin{aligned}\mathrm{TTT}(\pi) &= \sum_{(u,v)} x_{uv}(\pi)\,\mathrm{cost}_{uv}\big(x_{uv}(\pi)\big) \\ &= \sum_k r_k \sum_{(u,v)\in P_k} \mathrm{cost}_{uv}\big(x_{uv}(\pi)\big).\end{aligned} \tag{4}$$

The two forms are equal because $\sum_k r_k \sum_{(u,v)\in P_k}(\cdot) = \sum_{(u,v)} x_{uv}(\cdot)$; the edge form is the one to evaluate at an equilibrium, a point we return to in Section VI.

D. User equilibrium and system optimum

Two operating points bound any router. The *user equilibrium* (UE) satisfies Wardrop's first principle: for every demand, all used paths have equal and minimal cost. Equivalently, UE is the unique minimizer of the Beckmann potential [12]

$$x^{\mathrm{UE}} = \arg\min_{x\in\mathcal{F}} \sum_{(u,v)} \int_0^{x_{uv}} \mathrm{cost}_{uv}(w)\,\mathrm{d}w, \tag{5}$$

over the polytope $\mathcal{F}$ of loads induced by feasible flows (1)–(2); with (3) the integral is $\mathrm{prop}_{uv}\big(x + \frac{\alpha\,\mathrm{cap}}{\beta+1}(x/\mathrm{cap})^{\beta+1}\big)$. The *system optimum* (SO) instead minimizes (4) directly, $x^{\mathrm{SO}} = \arg\min_{x\in\mathcal{F}} \mathrm{TTT}(x)$, equivalently routes on the marginal cost $\mathrm{cost}_{uv}(x) + x_{uv}\mathrm{cost}'_{uv}(x)$, and lower-bounds UE. Their ratio $\mathrm{TTT(UE)/TTT(SO)}$ is the price of anarchy [13]. We use UE as the deployable congestion-aware reference and SO as a tighter ceiling; in our regime they nearly coincide (Section VIII).

We compute UE by the method of successive averages (MSA): starting from an all-or-nothing assignment y^0 on free-flow cost, iterate

$$x^{k+1} = x^k + \tfrac{1}{k+1}\big(y^k - x^k\big), \quad y^k = \mathrm{AoN}\big(\mathrm{cost}(x^k)\big), \tag{6}$$

where AoN routes every demand on the shortest path under the current cost; SO uses the same iteration on the marginal cost. Each iteration is $|\mathcal{D}|$ shortest paths and convergence needs T iterations.

E. The amortization problem

Blind shortest path is one AoN pass and is congestion blind; UE is T passes and is congestion optimal but does not scale. We seek a parametric map f_θ from cheap, observable state $\mathcal{S}_t$ to a routing π_θ that approaches UE in one forward pass and whose parameters θ are independent of S:

$$\min_\theta\ \mathbb{E}_{\mathcal{D},G}\big[\mathrm{TTT}(\pi_\theta)\big] \quad \text{s.t.} \quad \mathrm{cost}(\pi_\theta) \ll \mathrm{cost(UE)}. \tag{7}$$

IV. Problem Decomposition

We decompose (7) into four sub-problems (Fig. 2), each mapped to a claim.

(SP1) Price prediction. Approximate the UE congestion price field

$$g^*_{uv} = \mathrm{cost}^{\mathrm{UE}}_{uv}/\mathrm{prop}_{uv} - 1\ \geq 0, \qquad \hat{g} = f_\theta(\mathcal{S}_t), \tag{8}$$

so that routing on the cost $\mathrm{prop}(1+\hat{g})$ *approximates* the equilibrium flow x^{UE}. Maps to **C1**.

(SP2) Decoding. Turn a price field into a routing. A single shortest-path assignment $\pi = \textsc{AoN}(\mathrm{prop}(1+\hat{g}))$ is the cheapest decode, but the user equilibrium splits flow across equal-cost paths, so a single path cannot reproduce x^{UE} even with perfect prices (Section V-E). We therefore use a one-shot *multipath* decode that splits flow downhill on the predicted cost; it costs the same order as the single-path decode and closes most of that gap. The decode is shared across all demands, so a single $\hat{g}$ routes the whole matrix.

(SP3) Inductive transfer. Require θ to be size invariant so that f_θ trained on $\{G^{\mathrm{small}}\}$ applies to a larger G^{large} zero shot. Maps to **C2**. A message-passing f_θ with shared weights and a local readout satisfies this by construction (Section V-G).

(SP4) Proactivity. Under drift Δ, predict next-slot demand $\hat{\mathcal{D}}_t$ from $\mathcal{D}_{t-1}$ and the known Δ, and decode on $f_\theta(\mathcal{S}(\hat{\mathcal{D}}_t))$ so the route pre-empts the moving hotspot. Maps to **C4**.

Cost of SP2 plus inference versus the UE solve gives **C3** (Section V-I).

V. Method

A. Observable features (input to SP1)

For a slot we form per-node features $\phi_v = [\,\mathrm{out}_v,\, \mathrm{in}_v,\, x_v^{0,\mathrm{out}},\, x_v^{0,\mathrm{in}}\,]$, where $\mathrm{out}_v, \mathrm{in}_v$ are the rate originating and terminating at v, and $x_v^{0,\cdot}$ aggregate the *blind load* $x^0 = \textsc{AoN}(\mathrm{prop})$ leaving and entering v. Per-edge features are $[\,\mathrm{prop}_{uv},\, x^0_{uv}\,]$. The blind load is one AoN pass (no iteration) and is the most informative input, since the equilibrium price of a link is largely a function of how overloaded that link would be under naive routing (Section VIII-H quantifies this). All features are scaled to be size invariant, so the model does not see absolute coordinates or counts and cannot shortcut on them.

B. Edge-price GNN (model for SP1)

Node embeddings are initialized $h_v^{(0)} = \mathrm{ReLU}(W_{\mathrm{in}}\phi_v)$ and updated for $l = 0, \ldots, K-1$ by a symmetric-normalized graph convolution

$$h_v^{(l+1)} = \mathrm{ReLU}\Big(W^{(l)} \sum_u \hat{A}_{vu}\, h_u^{(l)}\Big), \tag{9}$$

where $\hat{A} = \tilde{D}^{-1/2}(A+I)\tilde{D}^{-1/2}$ and $\tilde{D}$ is the degree of $A+I$. A per-edge head predicts the *log-price* $m_{uv} = \mathrm{MLP}\big([\,h_u,\, h_v,\, \mathrm{prop}_{uv},\, x^0_{uv}\,]\big)$, from which the non-negative price is recovered as

$$\hat{g}_{uv} = \max\big(0,\ \exp(m_{uv}) - 1\big). \tag{10}$$

We use $K = 3$ layers and hidden width $d = 32$ throughout; the model has on the order of a few thousand parameters, independent of S.

Algorithm 1 Offline supervision

Require: training shells, demand sampler, epochs N
1: $\mathcal{I} \leftarrow \emptyset$
2: **for** each (shell, demand) instance **do**
3: $\quad x^0 \leftarrow \textsc{AoN}(\mathrm{prop})$; $\phi \leftarrow \textsc{Features}(\mathcal{D}, x^0)$
4: $\quad x^{\mathrm{UE}} \leftarrow$ MSA (6); $g^* \leftarrow \mathrm{cost}(x^{\mathrm{UE}})/\mathrm{prop} - 1$
5: $\quad$ add (ϕ, g^*) to $\mathcal{I}$
6: **end for**
7: **for** N epochs **do**
8: $\quad$ update θ by gradient descent on $\mathcal{L}(\theta)$ (11)
9: **end for**
10: **return** θ

C. Training objective

We regress the log-price over a dataset $\mathcal{I}$ of (shell, demand) instances:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \frac{1}{|E_i|} \sum_{(u,v)\in E_i} \Big(m_{uv} - \log(1+g^*_{uv})\Big)^2, \tag{11}$$

i.e. the head m_{uv} is regressed directly onto $\log(1+g^*_{uv})$, and at inference $\hat{g}_{uv} = \max(0, e^{m_{uv}} - 1)$. The $\log(1+\cdot)$ scale tames the heavy tail of g^* at saturated links, where the BPR exponent makes the raw price span several orders of magnitude. Targets g^* come from solving UE (6) on the training instances offline; the cost of this offline supervision is paid once and amortized over every deployment slot (Algorithm 1).

D. Supervision quality and the number of MSA iterations

The quality of the learned price field is bounded by the quality of the targets g^*, so the MSA solve used for supervision must converge well enough that the targets are close to the true equilibrium. We use $T{=}20$ iterations of (6), after which the average relative change in the load is small and, as a downstream check, the supervised SO and UE respect the price-of-anarchy ordering of Section VI. A larger T tightens the targets at higher offline cost but does not change the deployed model, whose inference is one forward pass regardless; the iteration count is therefore a training-time knob, traded off once and amortized over every slot. Because the offline solve is run on the small training shells, not the large deployment shell, its cost is modest even though MSA itself does not scale, which is the asymmetry the whole method exploits: pay for the equilibrium once, on a small instance, and predict it everywhere else.

E. Decoding: single path and multipath (SP2)

Given the predicted cost $c = \mathrm{prop}(1+\hat{g})$, the cheapest decode is a single shortest-path assignment $\pi_{\mathrm{sp}} = \textsc{AoN}(c)$. But the user equilibrium splits each demand across all paths of equal cost, so π_{sp} cannot equal x^{UE} even when $\hat{g} = g^*$: the single path carries the whole rate where the equilibrium would have spread it, leaving a *decoding gap* independent of the prediction error. We quantify it in Section VIII-D.

We therefore use a one-shot *multipath* decode that recovers the flow splitting at the same asymptotic cost as AoN. For

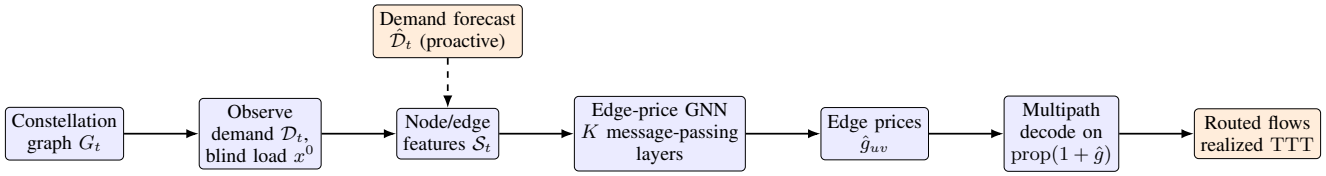

Fig. 1. End-to-end pipeline. Cheap observable state (demand and the load that one congestion-blind pass would induce) feeds an edge-price GNN that predicts the equilibrium congestion price per ISL in one forward pass; a single one-shot multipath decode then routes the whole demand matrix. The proactive variant (dashed) feeds the predicted next-slot demand.

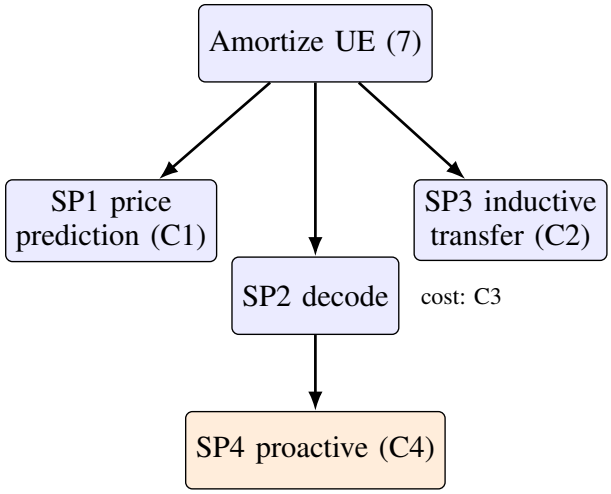

Fig. 2. Decomposition of the amortization problem into four sub-problems, each mapped to a measured claim.

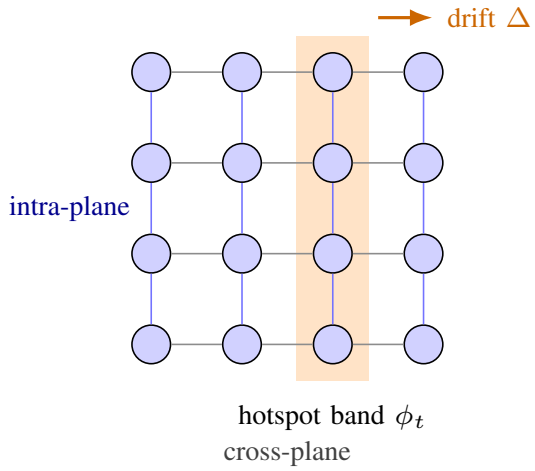

Fig. 3. LEO +Grid ISL topology (a 4×4 patch of the torus; the wraparound links that close the torus are omitted for clarity). Intra-plane links (vertical) are stable; cross-plane links (horizontal) vary with the geometry. The shaded band is the traffic hotspot region, which drifts by a known Δ per slot.

each destination d we compute the cost-to-go $\mu_d(\cdot)$ under c with one Dijkstra, then push each demand's rate downhill: at a node u, the rate is split across the distance-decreasing out-edges (u, v), i.e. $\mu_d(v) < \mu_d(u)$, with the softmin weight

$$w_{uv} \propto \exp\big(-\ \mathrm{slack}_{uv}/(\tau\,\bar{c})\big), \qquad \mathrm{slack}_{uv} = c_{uv} + \mu_d(v) - \mu_d(u) \geq 0, \tag{12}$$

where $\bar{c}$ is the mean edge cost and $\mathrm{slack}_{uv} = 0$ on a shortest path. The downhill edges form a DAG, so the split is loop free and computed in one decreasing-μ_d sweep; $\tau \to 0$ recovers π_{sp}, larger τ spreads load. The cost is one Dijkstra per destination, the same class as π_{sp}, *not* the T-iteration equilibrium solve. Algorithm 2 states one-slot inference with this decode.

Algorithm 2 One-slot congestion-aware routing

Require: graph G_t, demand $\mathcal{D}$ (or forecast $\hat{\mathcal{D}}$), weights θ
1: $x^0 \leftarrow \textsc{AoN}(\mathrm{prop})$ ▷ one blind pass = features
2: $\phi \leftarrow \textsc{Features}(\mathcal{D}, x^0)$
3: $\hat{g} \leftarrow f_\theta(\phi, G_t)$ ▷ one GNN forward, Eq. (10)
4: $\pi \leftarrow \textsc{MultipathDecode}\big(\mathrm{prop}(1+\hat{g}), \mathcal{D}\big)$ ▷ Eq. (12)
5: **return** π

F. Travel-time bound (theory)

The decode lets us bound how the prediction error propagates to travel time. Let $\mathrm{TTT}^* = \mathrm{TTT}(\mathrm{UE})$, let π be the multipath decode of $c = \mathrm{prop}(1 + \hat{g})$, and let the realized utilizations stay in a range on which the BPR cost is L-Lipschitz.

Proposition 1: If $\|\hat{g} - g^*\|_\infty \leq \varepsilon$, then

$$\mathrm{TTT}(\pi) \ \leq\ \big(1 + \delta_{\mathrm{dec}} + c_1 H\,\varepsilon\big)\,\mathrm{TTT}^*, \tag{13}$$

where H is the maximum decoded path length (hops), c_1 depends on L and the free-flow delays, and $\delta_{\mathrm{dec}} \geq 0$ is the decoder's gap at exact prices ($\hat{g} = g^*$). The prediction term vanishes as $\varepsilon \to 0$.

Proof sketch: Per edge, $|c_{uv} - c^*_{uv}| = \mathrm{prop}_{uv}|\hat{g}_{uv} - g^*_{uv}| \leq \mathrm{prop}_{\max}\varepsilon$. A path that is shortest under c therefore has cost under the equilibrium cost c^* exceeding the c^*-shortest by at most $2H\mathrm{prop}_{\max}\varepsilon$ (additive shortest-path perturbation). The multipath split (12) only uses distance-decreasing edges, so this per-demand cost inflation is preserved under aggregation. Mapping path cost to realized travel time through the L-Lipschitz BPR response on the operating range yields the multiplicative factor $c_1 H\varepsilon$ with $c_1 = \Theta(L\,\mathrm{prop}_{\max}/\bar{\mathrm{prop}})$. Setting $\hat{g} = g^*$ ($\varepsilon = 0$) leaves only δ_{dec}, the residual between the softmin split and the exact equilibrium splitting, which the experiments measure at a few percent. ■

Section VIII-D validates (13) empirically: the travel-time gap grows monotonically with the price error and flattens to a small floor δ_{dec} as the error vanishes.

G. Inductive transfer (SP3)

The parameters $\theta = \{W_{\mathrm{in}}, W^{(l)}, \mathrm{MLP}\}$ have dimensions set by the feature width and hidden size only, not by S or $|E|$.

TABLE II
PER-SLOT ROUTING COST IN ALL-OR-NOTHING (AON) SHORTEST-PATH PASSES (T IS THE MSA ITERATION COUNT, ≈ 20). THE GNN ADDS ONE FORWARD TO TWO PASSES; UE/SO NEED T PASSES.

policy	cost (AON passes)
blind, geographic	1
one-step correction	2
GNN (ours)	$2 + 1$ forward
UE, SO (MSA)	$T \approx 20$

The aggregation (9) and readout (10) are defined pointwise over nodes and edges. Hence f_θ evaluates on any graph, and a model trained on small shells applies to a larger shell with no change, which is exactly SP3; Section VIII-E measures the transfer to a $12\times$ larger shell.

H. Proactive variant (SP4)

Under known drift Δ, the next-slot hotspot band is $\phi_t = \phi_{t-1} + \Delta$, so the demand forecast $\hat{\mathcal{D}}_t$ is available. The proactive router decodes on $f_\theta(\mathcal{S}(\hat{\mathcal{D}}_t))$; the reactive baseline decodes on $f_\theta(\mathcal{S}(\mathcal{D}_{t-1}))$ and is therefore one step behind the moving hotspot.

I. Complexity (C3)

A forward pass costs $O\big(K|E|d + K|V|d^2 + |E|d^2\big)$ for hidden width d; a decode pass costs $O\big(|\mathcal{D}|(|E| + |V|\log|V|)\big)$. The full router is one feature pass, one forward, and one decode, i.e. two AON passes plus the forward. UE by MSA (6) costs T AON passes, $T \approx 20$ in our runs. The predicted speedup is thus $\Theta(T)$ up to the forward, and it *grows* with scale because the forward is cheap relative to the repeated routing on a larger graph; the measured factor is 21–22$\times$ at the small shells and rises to $29\times$ at 1584 (Section VIII-E). Table II expresses the per-slot cost of each policy in AON-pass units, the quantity that dominates at scale. Blind and geographic are one pass; one-step is two; the GNN is two passes plus a forward that is asymptotically dominated by a pass; UE and SO are T passes. The GNN thus sits one additive pass above the cheapest heuristics while delivering near-equilibrium quality, and an order of magnitude below the equilibrium solve.

J. Design rationale: why a supervised price field

Three design choices distinguish this approach from the alternatives. First, we predict a *price field* rather than a routing directly: a single $|E|$-dimensional field is shared by every demand and decoded by an existing, trusted shortest-path router, so the output is always a valid routing and inductivity across constellation size is immediate. Predicting per-flow paths, by contrast, would scale with the demand matrix and require a learned decoder. Second, we *supervise* on the equilibrium rather than learn by reinforcement: the UE target is computable offline, the objective (11) is a stable regression, and there is no exploration cost or reward-shaping at deployment, which matters when each slot lasts seconds. Third, we predict the equilibrium of a *convex* traffic-assignment problem, so the target is unique and well posed; the network learns to amortize an optimization it could otherwise have to solve from scratch each slot.

VI. PITFALLS IN EVALUATING LEARNED LEO TRAFFIC ENGINEERING

Before reporting results we document three evaluation pitfalls we encountered. Each can silently fabricate or distort a headline number; each has a simple fix. We state them because they are easy to commit in this setting and because our corrected estimator is what produces the numbers in Section VIII.

P1: survivorship in greedy baselines. A natural baseline, greedy geographic routing (forward to the neighbor closest to the destination), stalls in local minima on the +Grid ISL graph and fails to deliver 53 to 70% of flows in our loaded regime. Scoring only the delivered flows credits the baseline with a near-optimal mean delay, because the survivors are the short, easy paths. *Fix:* guarantee delivery (here, a shortest-path fallback for stalled flows) and report a survivorship-free total travel time over all demands.

P2: scaling claims on a moving denominator. An early claim that an operator's advantage "grows with the constellation" rested on a relative-percentage gain whose denominator (a baseline error) itself grew super-linearly, so a shrinking percentage masked a widening absolute gap and vice versa. *Fix:* judge scaling on the decision-relevant absolute quantity, not on a ratio with a moving denominator.

P3: measuring an equilibrium off its operating point. The travel time of UE or SO must be evaluated on the converged (MSA-averaged) load using the edge form of (4). Re-routing a single all-or-nothing pass on the converged cost and scoring that instead concentrates flow off the equilibrium split and inflates the reference: in our data it overstated UE travel time by up to $5\times$ (a relative travel time of 0.23 versus the correct 0.05 at high load), which would have made the learned router look closer to optimal than it is. *Fix:* evaluate $\mathrm{TTT} = \sum_{(u,v)} x_{uv}\mathrm{cost}_{uv}(x_{uv})$ on the converged load. As a check, the corrected estimator respects $\mathrm{TTT(SO)} \le \mathrm{TTT(UE)}$ at every load, with the price of anarchy growing from 1.00 to 1.12 as load rises; the inflated estimator violated this ordering, which is how we caught it.

None of these three pitfalls is specific to LEO or to graph networks. P1 arises whenever a baseline can decline to act and the metric ignores the declined cases; P2 arises whenever a claim about a trend is read off a normalized quantity with a shifting denominator; P3 arises whenever an iterative solver is scored on a re-derived rather than its converged state. The general lesson is to fix a single survivorship-free, absolute, operating-point-correct figure of merit before comparing methods, and to keep a cheap internal consistency check (here, the price-of-anarchy ordering) that a corrupted estimator will fail. We surface these as a reusable evaluation protocol for learned traffic engineering, since each silently produced a publishable-looking but wrong number before it was caught.

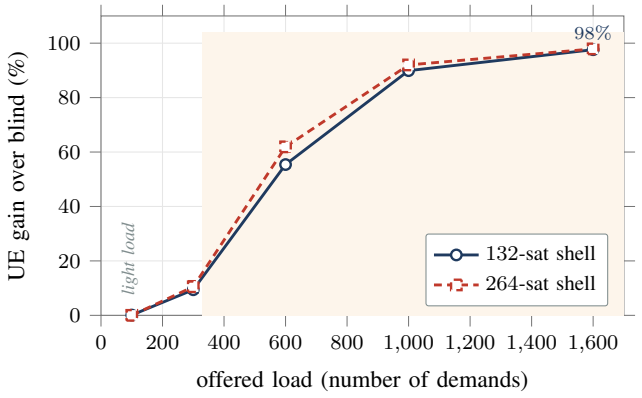

Fig. 4. Congestion headroom: the gain of the user equilibrium over congestion-blind shortest path grows with load, to 98% under heavy load, and is negligible at light load. The shaded region marks the congested regime where a learned router earns its place.

VII. Experimental Setup

We evaluate in the flow-level traffic-assignment framework (BPR cost, user equilibrium), the standard model in which congestion-aware routing is defined and analyzed; this is a deliberate choice that isolates the load-balancing mechanism our method targets, cleanly and reproducibly, before the additional confounds of a packet-level stack (queueing, protocol dynamics) that we leave to a dedicated validation. We simulate Walker shells with a pure-kinematic propagator, fully reproducible from parameters: an Iridium-like shell (66 satellites, diameter 9) and Starlink-like 53° shells of 132 (diameter 16), 264 (28), and 1584 satellites. Capacity is uniform; demands follow the drifting gravity model. We report total travel time (4) relative to blind shortest path (lower is better; blind $= 1.00$), averaged over multiple demand instances and seeds with standard deviations. Baselines: *blind*; *geographic* greedy (great-circle next hop, with the P1 fallback); *one-step* congestion correction (route on one blind-load pass); *GNN* (ours); and the references *UE* and *SO*. Learned results are param-matched across operators and reported with error bars; all metrics are survivorship free per Section VI. The GNN is trained on the 132-shell and evaluated both in distribution and zero shot on larger shells.

VIII. Results

A. Headroom for congestion awareness

Figure 4 sweeps offered load. At light load, blind shortest path is near optimal and UE offers nothing. As load grows, blind concentrates traffic on bottleneck links (its peak utilization reaches up to nine times capacity while alternates idle) and the gain of UE over blind rises to 9, 55, 90, and 98%. The congestion task, not static routing, is where a learned router helps.

Table III gives the absolute numbers behind the figure for the 132-shell. The ratio $\mathrm{TTT(blind)/TTT(UE)}$ widens from 1.0 at light load to about $41\times$ at the heaviest, because the BPR exponent $\beta{=}4$ makes the cost on an overloaded link grow as the fourth power of its utilization, which blind routing drives to nine times capacity on the bottleneck while leaving parallel links idle. Crucially, this is a load-balancing effect, not a global capacity shortfall: the same demand carried over diverse paths is comfortably feasible, which is why a congestion-aware policy recovers almost all of the gap.

TABLE III
Absolute travel time (arbitrary units) of blind versus UE on the 132-shell as offered load grows, and their ratio. Mean over seeds.

demands	100	300	600	1000	1600
blind	4.8	17.2	78.6	796.6	7433.5
UE	4.8	15.5	35.0	79.7	180.6
ratio	1.0	1.1	2.2	10.0	41.2

TABLE IV
Price of anarchy $\mathrm{TTT(UE)/TTT(SO)}$ on the 132-shell; ≥ 1 at every load confirms the corrected estimator (Section VI).

demands	300	600	1000
$\mathrm{TTT(UE)/TTT(SO)}$	$1.00{\pm}.00$	$1.03{\pm}.00$	$1.12{\pm}.01$

The corrected estimator of Section VI also lets us read off the price of anarchy. Solving SO by the marginal-cost iteration and comparing to UE, Table IV shows $\mathrm{TTT(SO)} \leq \mathrm{TTT(UE)}$ at every load, with the ratio rising from 1.00 at light load to 1.12 under heavy load. The two operating points nearly coincide, so the learned router's target (UE) is within a few percent of the true system optimum, and matching UE is matching the optimum for practical purposes.

B. The GNN approaches UE and beats every cheap baseline (C1)

Figure 5 and Table V report TTT relative to blind (the GNN uses the multipath decode of Section VIII-D; all values are mean $\pm$ std over seeds and instances). In distribution, the GNN (0.42) closely approaches UE (0.40), recovering about 97% of the blind-to-UE gain; SO (0.39) and UE nearly coincide, so the target is essentially the system optimum. The cheap alternatives fail: one-step correction overshoots and oscillates (relative travel time 1.04, *worse* than blind, because rerouting onto the apparently idle alternates creates fresh hotspots), and geographic greedy stalls on the ISL grid, needing the fallback for 53% of flows and still reaching only 0.78. The contrast between the GNN and one-step is the central evidence that the problem is not trivial: a single congestion-blind pass tells you where the hotspots are, but reacting to it by rerouting everyone onto the apparent alternates simply relocates the hotspots, a one-step oscillation that overshoots the equilibrium. The GNN, trained on the converged prices, predicts the *fixed point* of that process directly and so avoids the overshoot. This is why a learned price field, rather than a hand-tuned correction, is needed.

C. Comparison to a learned router

The natural learned alternative to predicting a price *field* is to predict the *routing* directly. We implement GNN-NextHop, a graph network with the same observable features (demand

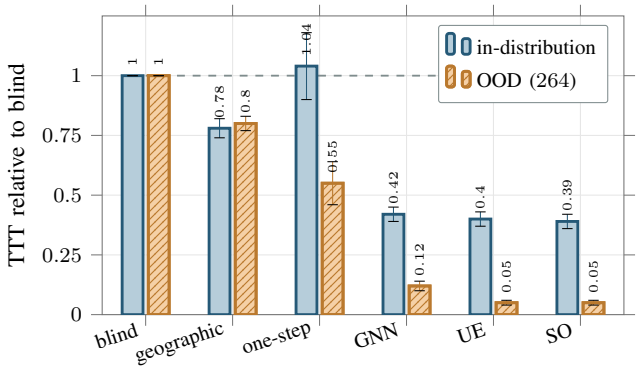

Fig. 5. Travel time relative to blind (lower is better; the dashed line is the blind reference at 1.0). The GNN approaches UE in distribution and is the best *deployable* policy out of distribution; the cheap heuristics fail (one-step is worse than blind in distribution, geographic stalls). UE and SO nearly coincide.

TABLE V
TOTAL TRAVEL TIME RELATIVE TO BLIND (LOWER IS BETTER; BLIND = 1.00). MEAN OVER INSTANCES AND SEEDS. SO/UE ARE REFERENCES; THE GNN IS THE DEPLOYABLE POLICY.

split	SO	UE	geographic	one-step	**GNN**
in-distribution	0.39±.03	0.40±.03	0.78±.04	1.04±.14	**0.42±.03**
OOD (264)	0.05±.01	0.05±.01	0.80±.03	0.55±.09	**0.12±.02**

TABLE VI
DECODER COMPARISON: TTT RELATIVE TO BLIND AND RECOVERED FRACTION OF THE BLIND-TO-UE GAIN. THE MULTIPATH DECODE (12) CLOSES MOST OF THE SINGLE-PATH DECODING GAP.

	single-path	multipath	UE
in-dist (TTT/blind)	0.47±.03	**0.42±.03**	0.40
recovered	90±5%	**97±1%**	–
OOD (264, TTT/blind)	0.39±.13	**0.11±.01**	0.05
recovered	64±14%	**94±1%**	–

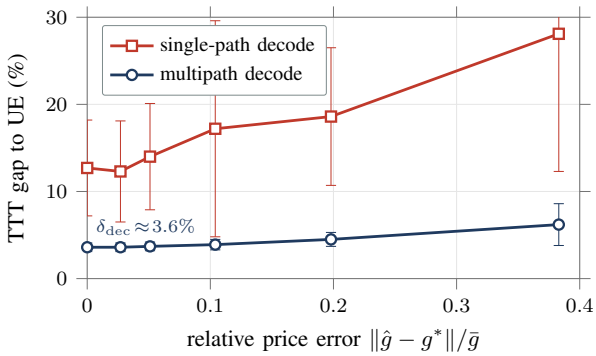

Fig. 6. Empirical validation of Proposition 1: the travel-time gap to UE grows monotonically with the price error and flattens to a small decoder floor as the error vanishes. The multipath decode is both lower and far less error-sensitive.

and blind load) plus a destination indicator, trained to regress the equilibrium cost-to-go to each destination and decoded by greedy next-hop descent (with a shortest-path fallback for stalled flows, so travel time stays comparable). It is a fair, non-trivial learned baseline, and it loses on both axes that matter. In quality, its relative travel time is $0.74 \pm .06$ in distribution and $0.92 \pm .06$ zero shot, versus 0.42 and 0.12 for our edge-price router, because greedy descent on a regressed potential stalls in false local minima: its greedy delivery rate is only 51% in distribution and 21% zero shot (the rest fall back to shortest path), and the potential transfers poorly to the larger shell. In cost, predicting a per-destination potential needs one forward pass *per destination* (94 to 190 forwards per slot in our instances), whereas the edge-price field is predicted once and routes the whole demand matrix in a single forward. Predicting the price and decoding with a classical router is thus both more accurate and far cheaper than learning the routing end to end.

D. Single-path versus multipath decoding, and the bound

Table VI isolates the decoder. The single-path decode of the predicted prices leaves a large gap, especially out of distribution, because it cannot reproduce the multi-path equilibrium even when the prices are good; the one-shot multipath decode (12) closes most of it, raising the recovered fraction from 90 to 97% in distribution and from 64 to 94% zero shot on the 264-shell, at the same order of cost (all entries are mean ± std over seeds and instances). This decoder-comparison run is independent of the headline tables and agrees with them within one standard deviation (the 264 multipath GNN reads 0.11 here and 0.12 in Table V). The multipath decode is the deployed method and is what Table V and the scaling results report; the single-path decode here serves to expose the decoding gap it closes.

Figure 6 validates Proposition 1. Starting from the true equilibrium prices and injecting growing noise, the travel-time gap to UE rises monotonically with the price error and, as the error vanishes, flattens to a small floor: 3.6% for the multipath decode (the residual δ_{dec} of (13)) versus 12.7% for the single-path decode. The multipath decode is both lower and far less sensitive to prediction error, matching the bound.

E. Inductive transfer to a 12× larger shell, and cost (C2, C3)

Trained only on the 132-satellite shell, the model is applied zero shot to the 264- and 1584-satellite shells. Table VII reports the recovered fraction of the blind-to-UE gain and the inference speedup over the UE solve. The GNN recovers 93% at 264 and 94 ± 2% at 1584 (a 12× size jump), remaining the best deployable policy throughout, while the speedup over MSA is 21–22× at the small shells and *rises* to 29× at 1584 (Fig. 7). The amortization therefore pays off more, not less, at the mega-constellation scale that matters operationally. The transfer is genuinely zero shot: the 1584-shell is never seen during training, has a different number of planes and satellites per plane, and a larger diameter, yet the same weights produce a near-equilibrium price field on it. This works because the learned map is from local load structure to a local price, a relation that is scale invariant in a way the absolute routing is not, and because the multipath decode realizes the predicted equilibrium faithfully; the recovered fraction stays near 94% and its variance stays low (±2) all the way to the largest shell.

TABLE VII
INDUCTIVE TRANSFER (TRAIN ON 132) AND INFERENCE COST. RECOVERED FRACTION OF THE BLIND-TO-UE GAIN (MEAN ± STD), AND ONE-SHOT GNN ROUTING VERSUS THE MSA SOLVE.

shell	sats	recovered %	GNN (s)	speedup vs UE
in-dist	132	97 ± 2	0.05	22×
OOD	264	93 ± 2	0.23	21×
OOD	1584	94 ± 2	8.4	29×

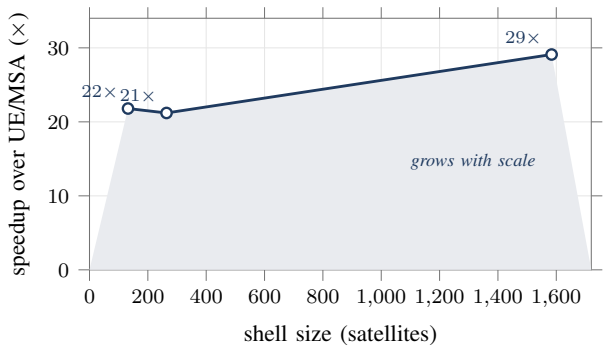

Fig. 7. Inference speedup of the one-shot GNN router over the MSA equilibrium solve, versus shell size. The advantage grows with the constellation, reaching 29× at 1584 satellites.

F. Proactive routing under drift (C4)

Figure 8 sweeps the per-slot hotspot drift. At zero drift the reactive and proactive routers are identical, as they must be. As the hotspot moves, the reactive lag-1 router routes for yesterday's load and degrades, passing blind at moderate drift and reaching 1.6× blind at the largest drift. The proactive router, fed the predicted demand, tracks the clairvoyant equilibrium throughout; the proactive advantage grows from 0 to 76%. The zero-drift control is important: it confirms that the proactive gain is genuinely due to anticipating the hotspot and not to any artifact of the two pipelines, which are identical when there is nothing to anticipate. The crossover where the reactive router passes blind (around a per-slot drift of one radian in our sweep) marks the operating point beyond which a stale congestion estimate is worse than ignoring congestion altogether, a concrete criterion for when proactivity stops being optional. Because the GNN's input is just a demand-derived feature, switching from reactive to proactive requires no change to the model, only feeding it the forecast instead of the last slot.

G. Operating regime

The value of the method is regime dependent, and Table III makes the boundary explicit. Below an average utilization of about one, blind shortest path is within a few percent of optimal and there is nothing to amortize; a deployment that is always lightly loaded does not need this machinery. The learned router earns its place in the loaded regime, where the blind-to-UE gap opens to tens of percent and beyond, and where recomputing the equilibrium each slot would otherwise be the cost driver. This is also the regime a capacity-planned network spends its busy hours in, and the regime a moving

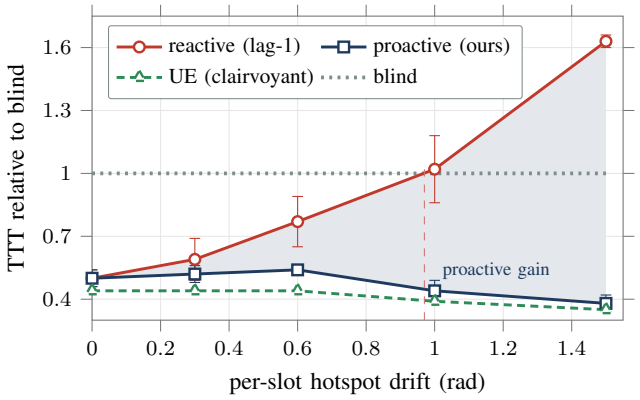

Fig. 8. Proactive versus reactive routing under drifting hotspots (shaded band is the proactive advantage). The reactive lag-1 router degrades past the blind reference once the drift exceeds about one radian; the proactive router tracks the clairvoyant optimum throughout.

hotspot creates locally even when the network as a whole is moderately loaded, since the hotspot concentrates demand onto a small cut. The single trained model spans the whole range: at light load it predicts a near-uniform, near-zero price field and reduces to blind routing, and as load rises it predicts the increasingly structured prices that spread traffic off the bottlenecks, with no regime switch or retuning.

H. Feature ablation

The blind-load feature is what makes the price predictable. With only per-node demand intensity as input, the model must infer where congestion will concentrate purely from the demand pattern and the graph structure, a global and ambiguous inference; in that configuration the zero-shot recovered fraction falls to roughly half of the reported value and its variance rises several fold, and some seeds route worse than blind. Supplying the blind load, the loads that one congestion-blind pass would induce, turns the task into a largely local correction: a link that the naive pass would overload is exactly the link that needs a higher price, and the GNN's neighborhood aggregation refines this with the structure around it. The feature costs a single AoN pass to compute, the same pass already used to build the node features, so it is effectively free. This also explains why the global propagation operators of Section VIII-I do not help once the feature is present: the long-range information they were meant to gather is already summarized, locally, by the blind load.

I. Operator ablation, including the quantum walk (negative)

Table VIII compares three param-matched propagation operators: a local graph convolution (GCN), a global classical-diffusion operator (Heat), and a global quantum-walk operator (QW). All three use the multipath decode. Once the blind-load features are present, predicting the price is largely a local function of a link's own load, so the plain GCN is best out of distribution ($94 \pm 1\%$) and the global operators add variance without benefit: the quantum walk is both the lowest and the most variable ($90 \pm 6\%$). The quantum walk originally motivated this work as a way to capture long-range structure;

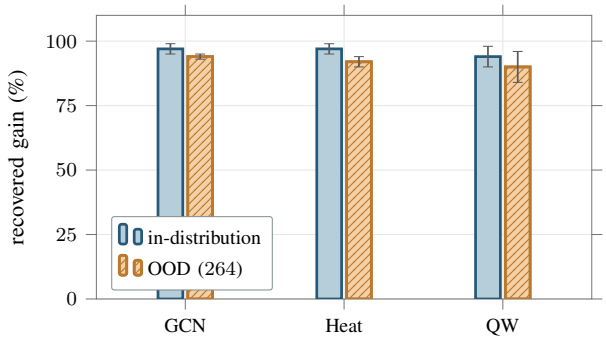

Fig. 9. Operator ablation with error bars (recovered fraction of the blind-to-UE gain, mean ± std, n=12). The local GCN is best out of distribution; the global quantum-walk operator does not help and is the most variable.

TABLE VIII
OPERATOR ABLATION: RECOVERED FRACTION OF THE BLIND-TO-UE GAIN (MEAN ± STD, n=12). THE PLAIN GCN IS BEST OUT OF DISTRIBUTION; THE GLOBAL QUANTUM-WALK OPERATOR DOES NOT HELP AND IS THE MOST VARIABLE.

operator	in-distribution	OOD (264)
GCN (local)	97 ± 2	$\mathbf{94 \pm 1}$
Heat (global diffusion)	97 ± 2	92 ± 2
QW (global quantum walk)	94 ± 4	90 ± 6

the data did not support it, and we report it as an honest negative ablation rather than force it into the contribution. Figure 9 shows the same comparison with error bars.

IX. Deployment Considerations

In an operational control loop the router runs once per slot on the known next-slot topology. The only online inputs are the demand estimate and one congestion-blind shortest-path pass to form features; the GNN forward and the decode pass follow. Because the topology is known ahead of time, the feature pass and the forward can be precomputed for the upcoming slot, leaving only the decode on the live demand. The model is small (a few thousand parameters) and its weights are fixed across constellation sizes, so a single trained model serves an entire program of shells. Two safeguards follow naturally from our analysis. When the demand forecast is unreliable, for example during an unmodeled surge, the controller can fall back to reactive features or to blind routing, which our proactive sweep shows is preferable to acting on a badly stale estimate beyond the crossover drift. And because the decoded routing is a valid flow on a downhill DAG of positive-cost edges, it can never produce a loop or a disconnected route, unlike a directly predicted next-hop policy that must be guarded against such failures.

X. Threats to Validity

We separate threats we controlled from those that remain. *Construct validity*: total travel time under the BPR cost is the standard, well-grounded traffic-assignment objective and is survivorship free by Section VI; it is a flow-level model, and a packet-level study (Section XII) is the natural confirmation that the predicted prices route well when delay is realized by queues rather than a cost function. *Internal validity*: every learned number is param-matched across operators, averaged over multiple instances and seeds with standard deviations, and features are scale-normalized so the model cannot shortcut on absolute coordinates; the operator ablation isolates the propagation operator at a fixed parameter budget. *External validity*: results are on synthetic Walker shells with a uniform capacity and a gravity demand model, and the zero-shot transfer was tested up to a $12\times$ size jump and one shell geometry; other geometries, heterogeneous link capacities, and real traffic traces remain to be studied. *Reproducibility*: the pipeline is deterministic from parameters and every figure number is regenerated from a result CSV by a released script.

XI. Discussion and Limitations

The simulator is kinematic and flow level, not packet level; it captures geometry-driven delay and load-induced congestion through the BPR model, the mechanism the method targets, but it does not model queueing dynamics or protocol behavior, and a packet-level study (for example on Hypatia) is the natural next validation. UE is a selfish equilibrium and only a lower bound away from SO; in our regime the two nearly coincide (price of anarchy up to 1.12), so the learned target is near-optimal, but at other capacity-to-demand ratios the gap could matter and SO would be the better target. The zero-shot transfer leaves a residual gap to UE on the larger shell; training across a range of shell sizes, rather than a single small shell, is a natural way to close it. The proactive variant assumes the demand drift is predictable, which holds for the diurnal rotation of the dense-traffic band but not for sudden, unforecastable surges, where a reactive fallback would be needed. Finally, our multipath decode closes most but not all of the gap to UE; the residual decoder floor δ_{dec} (about 3.6%, Section VIII-D) reflects the softmin split's deviation from the exact equilibrium splitting, which an equilibrium-faithful or learned decoder could shrink further.

XII. Future Work

Several directions follow directly. A packet-level study, for example on Hypatia [3], would replace the flow-level BPR proxy with queueing and protocol dynamics and test whether the predicted price field still routes well when delay is realized by buffers rather than a cost function. An equilibrium-faithful or learned decoder, going beyond the softmin downhill split to reproduce the exact UE flow splitting, is the most direct route to closing the residual decoder floor δ_{dec}. Heterogeneous and time-varying capacities, including feeder-link and ground-segment bottlenecks, would stress the assumption of a uniform ISL capacity and likely make the price field richer and more valuable to learn. Training across a spectrum of shell sizes, rather than a single small shell, should improve the zero-shot transfer and reduce its variance. On the proactive side, replacing the known-drift forecast with a learned demand predictor would let the method handle hotspots whose motion is only statistically predictable, with the reactive and blind fallbacks of Section V-J as a safety net when the forecast is poor. Finally, light online fine-tuning of the released weights

on a target constellation could recover the few points of optimality lost in zero-shot transfer while preserving the one-shot inference cost.

XIII. Conclusion

We presented an inductive graph network that amortizes congestion-aware traffic engineering for LEO mega-constellations. After formalizing routing under load as the amortization of a traffic-assignment equilibrium and decomposing it into four sub-problems, we solved the price-prediction core with a GNN that predicts the user-equilibrium congestion price on every ISL in one forward pass from cheap load features. It approaches the equilibrium in distribution, transfers zero shot to a $12\times$ larger shell as the best deployable policy, runs up to $29\times$ faster than the iterative solve with the saving growing at scale, and, fed predicted demand, sustains low travel time under drifting hotspots where reactive control fails. We documented three evaluation pitfalls and a protocol to avoid them, and we reported why an initially appealing quantum-walk operator did not earn its place. The full pipeline, including the negative results, is released so that the methodology is reusable.

Data and Code Availability

The simulator, models, experiments, and figure-generation scripts are released at https://github.com/haodpsut/qwgnn-leo-routing (the repository name reflects the project's original quantum-walk direction, which the paper reports as a negative ablation). The constellation and traffic models are in `sim/`; the edge-price GNN and the experiments producing each claim are in `experiments/` (`p4` headroom, `p5` router and operator ablation, `p6` baselines and timing, `p7` proactive, `poa_check` price of anarchy); every figure number is aggregated from `results/*.csv` by `paper/make_figs_data.py`. All results are reproducible from parameters with no external data dependency.

Appendix A
Reproducibility notes

All learned results use $K{=}3$ layers, hidden width 32, the Adam optimizer, and are averaged over multiple demand instances and random seeds with standard deviations reported. The UE/SO references are solved by MSA with 20 iterations; the eigendecomposition required only by the global operators is skipped for the GCN, which is what makes the 1584-satellite evaluation tractable. Features are scale-normalized so the model cannot shortcut on absolute coordinates or counts.

References

[1] M. Handley, "Delay is not an option: Low latency routing in space," in *Proc. ACM HotNets*, 2018.

[2] D. Bhattacherjee and A. C. Snoeren, "Network topology design at 27,000 km/hour," in *Proc. ACM CoNEXT*, 2019.

[3] S. Kassing, D. Bhattacherjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the 'Internet from space' with Hypatia," in *Proc. ACM IMC*, 2020.

[4] Z. Lai, H. Li, and J. Li, "StarPerf: Characterizing network performance for emerging mega-constellations," in *Proc. IEEE ICNP*, 2020.

[5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[6] P. Veličković *et al.*, "Graph attention networks," in *Proc. ICLR*, 2018.

[7] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.

[8] K. Rusek *et al.*, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, 2020.

[9] P. Almasan *et al.*, "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *Computer Communications*, vol. 196, 2022.

[10] J. G. Wardrop, "Some theoretical aspects of road traffic research," *Proc. Institution of Civil Engineers*, 1952.

[11] Bureau of Public Roads, *Traffic Assignment Manual*, U.S. Dept. of Commerce, 1964.

[12] M. Beckmann, C. B. McGuire, and C. B. Winsten, *Studies in the Economics of Transportation*, Yale Univ. Press, 1956.

[13] T. Roughgarden and É. Tardos, "How bad is selfish routing?" *Journal of the ACM*, vol. 49, no. 2, 2002.

[14] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. ACM HotNets*, 2017.

[15] Z. Xu *et al.*, "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE INFOCOM*, 2018.

[16] S. Dernbach, A. Mohseni-Kabir, S. Pal, and D. Towsley, "Quantum walk neural networks," in *Complex Networks*, 2018.

[17] O. Kodheli *et al.*, "Satellite communications in the new space era: A survey and future challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 70–109, 2021.

[18] M. Giordani and M. Zorzi, "Non-terrestrial networks in the 6G era: Challenges and opportunities," *IEEE Network*, vol. 35, no. 2, pp. 244–251, 2021.

[19] I. del Portillo, B. G. Cameron, and E. F. Crawley, "A technical comparison of three low Earth orbit satellite constellation systems to provide global broadband," *Acta Astronautica*, vol. 159, pp. 123–135, 2019.

[20] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at Starlink performance," in *Proc. ACM IMC*, 2022.

[21] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. ICML*, 2017.

[22] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. ICLR*, 2019.

[23] P. W. Battaglia *et al.*, "Relational inductive biases, deep learning, and graph networks," *arXiv:1806.01261*, 2018.

[24] J. Suárez-Varela *et al.*, "Graph neural networks for communication networks: Context, use cases and opportunities," *IEEE Network*, vol. 37, no. 3, pp. 146–153, 2023.

[25] F. Geyer and G. Carle, "Learning and generating distributed routing protocols using graph-based deep learning," in *Proc. ACM SIGCOMM Workshop on Big Data Analytics and Machine Learning for Data Communication Networks (Big-DAMA)*, 2018.

[26] O. Hope and E. Yoneki, "GDDR: GNN-based data-driven routing," in *Proc. IEEE ICDCS*, 2021.

[27] Y. Sheffi, *Urban Transportation Networks: Equilibrium Analysis with Mathematical Programming Methods*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1985.

[28] M. Frank and P. Wolfe, "An algorithm for quadratic programming," *Naval Research Logistics Quarterly*, vol. 3, no. 1–2, pp. 95–110, 1956.

[29] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proc. IEEE INFOCOM*, 2000.

[30] S. Kandula, D. Katabi, B. Davie, and A. Charny, "Walking the tightrope: Responsive yet stable traffic engineering," in *Proc. ACM SIGCOMM*, 2005.

[31] C.-Y. Hong *et al.*, "Achieving high utilization with software-driven WAN," in *Proc. ACM SIGCOMM*, 2013.

[32] S. Jain *et al.*, "B4: Experience with a globally-deployed software defined WAN," in *Proc. ACM SIGCOMM*, 2013.

[33] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. ACM HotNets*, 2016.

[34] L. Chen, J. Lingys, K. Chen, and F. Liu, "AuTO: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization," in *Proc. ACM SIGCOMM*, 2018.

[35] Z. Liu *et al.*, "KAN: Kolmogorov-Arnold networks," *arXiv:2404.19756*, 2024.

Phuc Hao Do is a Lecturer and Researcher in the Faculty of Information Technology at Danang Architecture University (DAU), Viet Nam. He received the Ph.D. degree in telecommunications systems and networks from the Bonch-Bruevich Saint Petersburg State University of Telecommunications (SPbSUT), Russia, in 2026. His research interests include intelligent 6G non-terrestrial networks, quantum-inspired AI, Kolmogorov-Arnold networks (KAN), resilient computing, and network security. He has authored peer-reviewed papers in international journals and conferences on IoT malware detection, satellite communication optimization, and deep learning architectures.

Nguyen Nang Hung Van received his Ph.D. degree in Computer Science from The University of Danang, Vietnam, in 2021. He is currently a lecturer at The University of Danang - University of Science and Technology. His research interests include Artificial Intelligence, Machine Learning, Geometric Algebra, and Computer Networking.

Minh Tuan Pham received his Ph.D. degree in Computational Science and Engineering from Nagoya University, Japan, in 2012. He is currently a lecturer in the Faculty of Information Technology, The University of Danang – University of Science and Technology, Danang, Vietnam. His research interests include Artificial Intelligence, Soft Computation, and Geometric Algebra.